

神经网络与反向传播: 正向传播→计算损失→反向传播
神经网络-解决线性不可分问题 参数更新

常见激活函数:

$\text{sigmoid}: \sigma(x) = \frac{1}{1+e^{-x}}$ Leaky ReLU: $\max(0, x, x)$
 $\tanh: \tanh(x) = \frac{e^x - 1}{e^x + 1}$ Maxout: $\max(w_1^T x + b_1, w_2^T x + b_2)$
ReLU(默认): $\max(0, x)$ ELU: $\begin{cases} x, & x \geq 0 \\ \alpha(e^x - 1), & x < 0 \end{cases}$

正向传播步骤: ①线性组合求权重 ②代入激活函数计算

线性组合结果: $z = w_1 a_1 + w_2 a_2 + \dots + w_n a_n + b$ $a = g(z)$
偏置 x_i 神经网络输出

②正向传播相减的标准流程: 先线性求和, 再带函数求。

①回归问题: 激活函数 $g(x) = x$ ②二分类 sigmoid

③多分类 Softmax: 先将输入指数化, 再求和。

$[z_1, z_2, \dots, z_n] \rightarrow [e^{z_1}, e^{z_2}, \dots, e^{z_n}] \rightarrow \frac{e^{z_i}}{\sum e^{z_j}}$ 再将输入全归一化, 和为1。

计算损失步骤: 前向传播, 给定损失函数 L , 用 L 求偏导。

反向传播步骤: 上游梯度 $\times$ 本地梯度 = 下游梯度 [链式]

梯度计算的最终目标: $\frac{\partial L}{\partial w}$ 核心自归。

e.g. 求 $f(x_0, x_1, w_0, w_1, w_2) = \frac{1}{1 + \exp(-(w_0 x_0 + w_1 x_1 + w_2))}$ 的偏导 $\frac{\partial f}{\partial x_0}, \frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial w_0}, \frac{\partial f}{\partial w_1}, \frac{\partial f}{\partial w_2}$ 。基于节点进行梯度下降。

再加上 J 就是完整的反向传播过程。

梯度传播由上游梯度对值本地梯度乘得。

节点 J 的完整反向传播过程。

节点 J 的完整反向传播过程。

节点 J 的完整反向传播过程。

节点 J 的完整反向传播过程。

节点 J 的完整反向传播过程。

节点 J 的完整反向传播过程。

节点 J 的完整反向传播过程。

节点 J 的完整反向传播过程。

节点 J 的完整反向传播过程。

节点 J 的完整反向传播过程。

节点 J 的完整反向传播过程。

节点 J 的完整反向传播过程。

节点 J 的完整反向传播过程。

节点 J 的完整反向传播过程。

节点 J 的完整反向传播过程。

节点 J 的完整反向传播过程。

节点 J 的完整反向传播过程。

节点 J 的完整反向传播过程。

机器学习基础: 分类/回归问题: 用分类的方法

解回归问题: 先求梯度, 再求参数。

欠拟合: 训练误差小, 测试误差大。

过拟合: 训练误差大, 测试误差小。

最近邻算法: Simplist: 对测试样本, 用训练样本中距离最近的 k 个样本中多数的标签来预测。

缺点: 完全遍历耗时, 距离函数复杂: $d = \sqrt{(x_1 - x_2)^2 + (x_2 - x_3)^2 + \dots}$

优点: 不用训练, 随拿随用。

线性回归: $y = w^T x + b$

训练方法: 使用平方损失函数 L 和梯度下降。

训练误差: $L(y_i, \hat{y}_i) = \frac{1}{2}(y_i - \hat{y}_i)^2$

训练误差: $J(w, b) = \frac{1}{2} \sum (y_i - \hat{y}_i)^2$

训练误差: $J(w, b) = \frac{1}{2} \sum (y_i - \hat{y}_i)^2$

训练误差: $J(w, b) = \frac{1}{2} \sum (y_i - \hat{y}_i)^2$

训练误差: $J(w, b) = \frac{1}{2} \sum (y_i - \hat{y}_i)^2$

训练误差: $J(w, b) = \frac{1}{2} \sum (y_i - \hat{y}_i)^2$

训练误差: $J(w, b) = \frac{1}{2} \sum (y_i - \hat{y}_i)^2$

训练误差: $J(w, b) = \frac{1}{2} \sum (y_i - \hat{y}_i)^2$

训练误差: $J(w, b) = \frac{1}{2} \sum (y_i - \hat{y}_i)^2$

训练误差: $J(w, b) = \frac{1}{2} \sum (y_i - \hat{y}_i)^2$

训练误差: $J(w, b) = \frac{1}{2} \sum (y_i - \hat{y}_i)^2$

训练误差: $J(w, b) = \frac{1}{2} \sum (y_i - \hat{y}_i)^2$

训练误差: $J(w, b) = \frac{1}{2} \sum (y_i - \hat{y}_i)^2$

训练误差: $J(w, b) = \frac{1}{2} \sum (y_i - \hat{y}_i)^2$

训练误差: $J(w, b) = \frac{1}{2} \sum (y_i - \hat{y}_i)^2$

训练误差: $J(w, b) = \frac{1}{2} \sum (y_i - \hat{y}_i)^2$

训练误差: $J(w, b) = \frac{1}{2} \sum (y_i - \hat{y}_i)^2$

训练误差: $J(w, b) = \frac{1}{2} \sum (y_i - \hat{y}_i)^2$

训练误差: $J(w, b) = \frac{1}{2} \sum (y_i - \hat{y}_i)^2$

训练误差: $J(w, b) = \frac{1}{2} \sum (y_i - \hat{y}_i)^2$

训练误差: $J(w, b) = \frac{1}{2} \sum (y_i - \hat{y}_i)^2$

训练误差: $J(w, b) = \frac{1}{2} \sum (y_i - \hat{y}_i)^2$

训练误差: $J(w, b) = \frac{1}{2} \sum (y_i - \hat{y}_i)^2$

训练误差: $J(w, b) = \frac{1}{2} \sum (y_i - \hat{y}_i)^2$

训练误差: $J(w, b) = \frac{1}{2} \sum (y_i - \hat{y}_i)^2$

训练误差: $J(w, b) = \frac{1}{2} \sum (y_i - \hat{y}_i)^2$

训练误差: $J(w, b) = \frac{1}{2} \sum (y_i - \hat{y}_i)^2$

训练误差: $J(w, b) = \frac{1}{2} \sum (y_i - \hat{y}_i)^2$

多分类-Softmax回归: 类别越多越好, 二分类共同训练 K

模型 $f(x) \in R^K$, 将 $f(x)$ 表示为取第 K 类概率 $\text{Softmax}(f(x))$

$P(y=k|x) = \frac{e^{f_k(x)}}{\sum_{j=1}^K e^{f_j(x)}}$ 满足归一化概率和为1

最大似然估计: $\max \rightarrow \sum \log \left[\frac{e^{f_k(x_i)}}{\sum_{j=1}^K e^{f_j(x_i)}} \right]$

最小交叉熵损失: $\min \rightarrow \sum \left[-\log \left(\frac{e^{f_k(x_i)}}{\sum_{j=1}^K e^{f_j(x_i)}} \right) - f_k(x_i) \right]$

正则化-防止过拟合: 防止过拟合/无用维度干扰

$\min \rightarrow \sum L(f(x_i), y_i) + \lambda \sum w^2$ 正则化 (L2)

①特征维度太多: 权重过大? $R(f) = \|w\|_2^2 = \sum w_j^2$, 放大看, 使大看的方向更多

②无特征维度太多: 权重 $R(f) = \|w\|_1 = \sum |w_j|$, 使稀疏, 回归系数 λ 因

选择: 超参数, 模型交叉验证:

交叉熵函数: α 学习率, λ 正则化

广义超参数: 使用什么模型/损失函数/正则化。

验证集: 训练, 验证: 测试 = 8:1:1

验证集: 训练, 验证: 测试 = 8:1:1

K折交叉验证: 10% 验证集, 有时过? $\rightarrow$ 取 Average

$\rightarrow$ 将训练集数据分成 K 份, 每次用 $K-1$ 份训练, 1 份验证。

$\rightarrow$ 遍历所有超参数组合, 取平均误差最小的组合。

决策树与随机森林结合得到公式应用:

定义: 常用机器学习方法, 树形结构, 根节点包含全集

每个非叶子节点对应一个属性二分, 每个叶子节点对应决策

结果 (取多数类) 作为决策结果 $\rightarrow$ 理想: 叶子节点包含纯净

样例: $\{1, 2, 3, 4, 5, 6, 7, 8, 9\}$

样例: $\{1, 2, 3, 4, 5, 6, 7, 8, 9\}$

样例: $\{1, 2, 3, 4, 5, 6, 7, 8, 9\}$

样例: $\{1, 2, 3, 4, 5, 6, 7, 8, 9\}$

样例: $\{1, 2, 3, 4, 5, 6, 7, 8, 9\}$

样例: $\{1, 2, 3, 4, 5, 6, 7, 8, 9\}$

样例: $\{1, 2, 3, 4, 5, 6, 7, 8, 9\}$

样例: $\{1, 2, 3, 4, 5, 6, 7, 8, 9\}$

样例: $\{1, 2, 3, 4, 5, 6, 7, 8, 9\}$

样例: $\{1, 2, 3, 4, 5, 6, 7, 8, 9\}$

样例: $\{1, 2, 3, 4, 5, 6, 7, 8, 9\}$

样例: $\{1, 2, 3, 4, 5, 6, 7, 8, 9\}$

样例: $\{1, 2, 3, 4, 5, 6, 7, 8, 9\}$

样例: $\{1, 2, 3, 4, 5, 6, 7, 8, 9\}$

样例: $\{1, 2, 3, 4, 5, 6, 7, 8, 9\}$

样例: $\{1, 2, 3, 4, 5, 6, 7, 8, 9\}$

数学基础:

1. 偏导数与链式法则: 对

$z(x, y) = f(u(x, y), v(x, y))$ 求 x, y 的偏导

$\Rightarrow \frac{\partial z}{\partial x} = \frac{\partial f}{\partial u} \frac{\partial u}{\partial x} + \frac{\partial f}{\partial v} \frac{\partial v}{\partial x}$

2. 标量对向量求导: 得到梯度 (向量)

设 $y = f(x) = f(x_1, \dots, x_n)$, $x = [x_1, \dots, x_n]^T$

则 $\frac{\partial y}{\partial x} = \left[\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right]^T$ 即 $\nabla f(x)$

3. 标量对矩阵求导: 得到矩阵

若 $X = \begin{bmatrix} x_{11} & \dots & x_{1n} \\ \vdots & \ddots & \vdots \\ x_{m1} & \dots & x_{mn} \end{bmatrix}$, 则 $\frac{\partial y}{\partial X} = \begin{bmatrix} \frac{\partial y}{\partial x_{11}} & \dots & \frac{\partial y}{\partial x_{1n}} \\ \vdots & \ddots & \vdots \\ \frac{\partial y}{\partial x_{m1}} & \dots & \frac{\partial y}{\partial x_{mn}} \end{bmatrix}$

Δ 矩阵求导有利于 GPU 并行加速。

条件熵与联合熵关系:

$H(X, Y) = H(X) + H(Y|X) = H(Y) + H(X|Y)$

互信息: $I(X, Y) = H(X) + H(Y) - H(X, Y)$

$H(X|Y) = H(X, Y) - H(Y)$

$H(Y|X) = H(X, Y) - H(X)$

$I(X, Y) = H(X) - H(X|Y) = H(Y) - H(Y|X)$

$I(X, Y) = H(X) + H(Y) - H(X, Y)$

$I(X, Y) = H(X) + H(Y) - H(X, Y)$

$I(X, Y) = H(X) + H(Y) - H(X, Y)$

$I(X, Y) = H(X) + H(Y) - H(X, Y)$

$I(X, Y) = H(X) + H(Y) - H(X, Y)$

$I(X, Y) = H(X) + H(Y) - H(X, Y)$

$I(X, Y) = H(X) + H(Y) - H(X, Y)$

$I(X, Y) = H(X) + H(Y) - H(X, Y)$

$I(X, Y) = H(X) + H(Y) - H(X, Y)$

$I(X, Y) = H(X) + H(Y) - H(X, Y)$

$I(X, Y) = H(X) + H(Y) - H(X, Y)$

$I(X, Y) = H(X) + H(Y) - H(X, Y)$

$I(X, Y) = H(X) + H(Y) - H(X, Y)$

$I(X, Y) = H(X) + H(Y) - H(X, Y)$

$I(X, Y) = H(X) + H(Y) - H(X, Y)$

$I(X, Y) = H(X) + H(Y) - H(X, Y)$

$I(X, Y) = H(X) + H(Y) - H(X, Y)$

$I(X, Y) = H(X) + H(Y) - H(X, Y)$

$I(X, Y) = H(X) + H(Y) - H(X, Y)$

$I(X, Y) = H(X) + H(Y) - H(X, Y)$

$I(X, Y) = H(X) + H(Y) - H(X, Y)$

属性:从离散到连续:

离散化技术=二分法:取点=直插+/-

衡量=二分点选取的好坏?选取信息增益最大的

数据的标签如果是连续的??

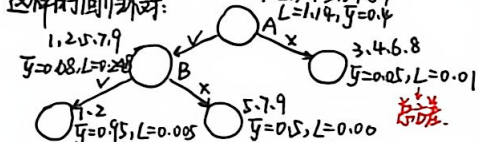
将上表y转为连续排列分析:从离散到连续

num	A	B	C	y
1	✓	✓	✓	0.1
2	✓	✓	✓	0.2
3	✓	✓	✓	0.3
4	✓	✓	✓	0.4
5	✓	✓	✓	0.5
6	✓	✓	✓	0.6
7	✓	✓	✓	0.7
8	✓	✓	✓	0.8
9	✓	✓	✓	0.9

求 $L(D,A)$ 5: 1.0, 9.0, 5.0, 5.0, 5.0, 5.0, 5.0, 5.0, 5.0

$L(D,A) = (1-0.68)^2 + 0.68^2 + 3 \times (0.5-0.68)^2 + 2 \times (0.1-0.68)^2 + 2 \times (0.9-0.68)^2 = 0.258$

这样的回环对:



总结:越大越好①增益②信息增益

越小越好①基尼指数②方差

集成学习+随机森林:

集成学习=多个个体学习器+结合策略(列举森林)

个体学习器:线性模型或决策树

随机森林:以决策树为个体学习器

以样本扰动+属性扰动为结合策略

Steps: ①样本扰动:对每个决策树,对训练集随机采样得到一个独立训练集

②属性扰动:对每个决策树,从训练集采样的一部分样本特征进行子树划分训练

③最后将所有训练好的决策树的平均预测(多类)作为对测试样本输出

BFS → queue 队列实现 → 先进先出,依次检查

DFS → stack 栈实现 → 后进先出,回溯求解

UCS: 一致代价搜索,关注累进代价

开闭表:使用总成本优先队列,扩展代价较低

UCS完备性:有限制恒即可找到/局限:没有目标方向

A*算法:较BFS,DFS,UCS更有方向性!!

①启发搜索:引入函数f与目标距离(例:曼哈顿距离)

②贪心搜索:永远搜索起来最近节点但不一定是最优的

③A*搜索:混合启发函数+贪心搜索

核心:估价函数 $f(n)=g(n)+h(n)$

估价函数:从节点n到目标点的实际代价

A*算法前提:可找到

最短路径的前提:启发函数 $h(n)$ Admissible

A*算法首次遇到目标即最优且不重复

展的前提:启发函数 $h(n)$ Consistent

Admissible: $h(n) \leq h^*(n)$

Consistent: $h(n) \leq C(n,n') + h(n')$

其中 $C(n,n')$ 是节点n到节点n'的实际代价

Consistent证明:前边即最优且不重复

1-假设A*已经扩展了一个节点n,但后来发现一条更短的路径到达n,那么这条路径上的某个节点m在n被首次扩展时未扩展

2-利用一致性可知,从起始节点到n的路径上,f值不会增加,因此,如果存在更短的g(n),则用那条更短路径上的某个节点会有更小的f值,应优先扩展

总结:若h一致Consistent,则一旦节点被扩展,其g(n)已经是最小值,之后不可能再有更小的f值出现

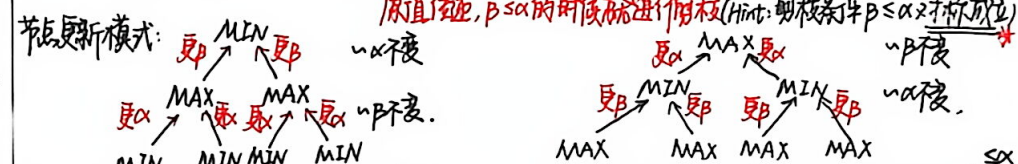
⇒ Admissible 可采纳性则保证了全局最优路径是不存在的

⇒ 而Consistent 一致性保证了主路路径的f值都是不存在的,更加偏向A*搜索

启发函数 $h(x)$ 如何设计需思考研究

常用启发函数-曼哈顿距离

Minimax搜索 & Alpha-Beta 剪枝:一句话概括:MAX节点更新传入的 α ,MIN节点更新传入的 β ,另外个原值比较, $\beta < \alpha$ 的时候就进行剪枝(Hint:剪枝条件 $\beta < \alpha$ 又对称成立)



在MAX节点,如果某个子MIN节点返回的值已大于 β ,那么父MAX节点就不会选择这个子MIN节点所在路径,因为父节点已经有另一个子节点的值 $\geq \beta$,这里 β 的值!因此父MAX节点中,一旦 $\alpha \geq \beta$,则后面子节点剪去!

在MIN节点,如果某个子MAX节点返回的评估值 $\leq \alpha$,那么父MIN节点就不会选择这个子MAX节点所在路径,因为父节点已经有另一个子节点的值 $\leq \alpha$,此处 α 不被剪!因此父MIN节点中,一旦 $\beta \leq \alpha$,后面子节点剪去!

Minimax-AB的一些数学性质:

① Minimax: 1段段双方完美理性,效率等于DFS,完整搜索数过大

② AB剪枝的优化程度:若子节点数排序,可解深度会 $\times 2 \rightarrow$ Max全搜而min可全被剪(后发制人)

③ 估值函数永远是不完美的

CSP搜索-约束满足问题(例:图形着色)~在约束域内找解

使用回溯搜索:改进-①每层只考虑一个变量②每步都考虑约束满足情况 $\rightarrow$ x, y, z 逐一一代0/1测试

逐层检查提前剪枝

从CSP到SAT问题 $\rightarrow$ 是否存在布尔(0/1)取值组合使所有逻辑约束可满足

使用DPLL算法较CSP回溯更高效

DPLL子句算法:子句内用或连接,子句间用与连接(合取范式判断子句)

在子句中依次设置变量0/1,以此句为假则回溯

子句状态:①已满足②无法满足③单字句 $\rightarrow$ 除了一个子句条件其他均为假 $\Rightarrow$ 则该状态必减为1,达剪枝

子句升级 DPLL $\rightarrow$ CDCL(子句的)子句学习算法

在子句的起点与终点中,一个假分式,两边和不同区域 $\rightarrow$ 相当于到了一个新的子句

$\Rightarrow$ 学习子句:划分多种多样,得到平衡的子句更利于搜索 $\rightarrow$ 又新的"子句的新子句"应在回溯后起学习子句,以剪枝

MCTS:蒙特卡洛树搜索,平衡"利用与探索"选当下最优的/当下非最优的

数学语言formulate: ①K:动作数 ② q^a :动作a的真实价值 ③ $Q(t,a)$:t时刻对a的估计值 $\rightarrow$ 模拟对局完成!!

④ $N(t,a)$:t时刻之前a被选中次数 ⑤ R_t :在t时刻得到价值 ⑥ A_t :在t时刻采用动作

平衡性有保证:以估计值 $Q(t,a) = \frac{\sum_{s=1}^t R_s \cdot \mathbb{1}_{A_s=a}}{N(t,a)}$ 故:设 ϵ , 1- ϵ 时选当前最优

极端值 $\rightarrow$ 概率收敛到最优, 0, $N(t,a)=0$

进一步 ϵ -greedy: UCB $\Rightarrow$ 将随机性隐含在探索于模拟随机结果中

$\Rightarrow$ 表示探索次数 $N(t,a)$ 少的节点要优先探索,其中C为探索的惩罚: $A_t = \arg \max [Q(t,a) + C \sqrt{\frac{\ln t}{N(t,a)}}]$

累路总结:①构建向前看的博弈树②随机模拟产生估值函数 $\Rightarrow$ 即时 $Q(t,a) + C \sqrt{\frac{\ln t}{N(t,a)}}$ 的最大

③在博弈树的中间节点通过UCB选择动作

MCTS循环:选择——扩展——模拟——回溯 $\rightarrow$ 节点越来越多,形成不对称的搜索树

UCB每次判断,直到根节点后,根节点模拟模拟 $\rightarrow$ 更新全路估值函数(胜率函数与估值)

(注意上边链接) ①单节点②子树扩展